Reading What Matters: Table-Conditioned Text Reading for Text-Tabular Prediction

Daniel Polyanovsky
Technion – Israel Institute of Technology

Abstract

Text-tabular models embed every text field of a row with an encoder that ignores the row's structured columns. We make reading *table-conditioned* at two levels. A table encoder initialises a recurrent *anytime* reader that consumes field-tagged sentence items one at a time, and an expected-gain head, adapted from dynamic feature selection, decides which items to read; *TabLoRA*, a low-rank adapter in E5's top layers whose rank space is gated per row by the table encoding, changes how each item is encoded. On all 20 MulTaBench text datasets (five splits each), learned reading beats a random reader with the same per-row budget at every budget we test (+0.026 AUROC and +0.051 R^2 at a nominal 10% of the items). For full accuracy, cross-fitted TabLoRA readers blended with Joint-Frozen TabM and TabPFN-2.5 reach 0.913 AUROC and 0.738 R^2, significantly above our earlier CatBoost blend (+0.013 AUROC, +0.036 R^2) and Joint-Frozen TabM on identical rows, and at the level of the MulTaBench Joint TAR references (TabM TAR: 0.911 and 0.728).

1 Introduction

In text-tabular learning, a row combines numerical and categorical columns with free-text fields such as reviews or job descriptions [1]. Standard practice embeds each field with a pretrained encoder such as E5 [21] and trains a tabular learner on the result (*Joint Frozen*), or fine-tunes the encoder on the target (*Joint TAR*) [2, 1]. Either way, the text is encoded without looking at the rest of the row, and every field is treated as equally necessary. Human analysts read differently: if a wine's price and region already pin down its grade, the review matters only when the table leaves the answer open, and the same sentence ("a bargain") means something different for a $9 and a $90 bottle. We build both observations into one model: the structured part of a row initialises a recurrent reader that acquires text one *item* (a field-tagged sentence chunk) at a time, a gain head decides which item is worth reading next, and the encoder's own adapters are modulated by the table.

Contributions. (i) A table-conditioned anytime reader for text-tabular rows, valid after any subset of reads, with a label-free reading policy that regresses observed one-step loss reductions, in the spirit of DIME [6]. (ii) TabLoRA: LoRA [12] adapters in E5 whose rank space is gated by the row's table encoding, trained through the reader. (iii) A leakage-free combination of the reader with strong tabular learners: cross-fitted reader predictions, blended with TabM [7] and TabPFN-2.5 [9] on validation rows. (iv) An evaluation on all 20 MulTaBench text datasets with paired local baselines on identical rows and ablations of each part.

2 Related Work

Text-tabular learning. MulTaBench [1] curates datasets on which text and structure are jointly needed and target-aware representations help. It evaluates CatBoost [16], LightGBM [13], TabM [7]

NLP 097215 Final Project (Track 1), Technion, 2026. Code: `https://github.com/htmldanielpoly/nlp_final_project`; weights: `https://huggingface.co/htmldaniel/reading-agent-v2-multabench`.

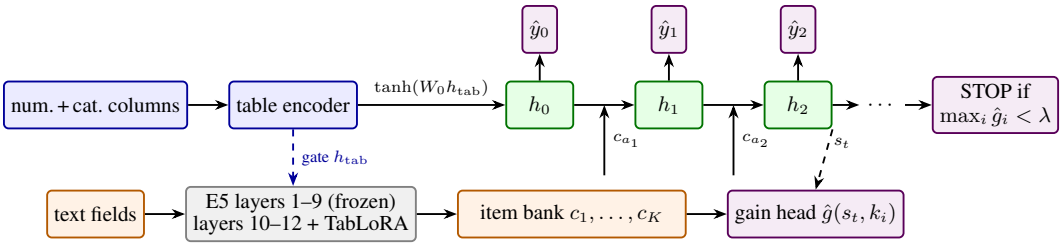


Figure 1: The reader: the table initialises a 2-layer GRU and gates the LoRA adapters of E5's top layers; at each step the gain head scores unread items, the reader consumes the best one, and $\hat{y}_t$ is available after every step.

and TabPFN [10, 9] with frozen or LoRA-tuned [12] E5 embeddings; TabSTAR [2] unfreezes the text encoder inside a tabular foundation model, and CARTE [14] embeds column names and strings. Ensembling neural text models with tabular learners is strong [19]; we combine them by stacking-style cross-fitting [22]. **Conditioning and adaptive reading.** TabLoRA's gate is a FiLM-style [15] scaling of LoRA's rank space by side information. Dynamic feature selection chooses features per instance [20, 4]; DIME [6] regresses the loss reduction and stops at a threshold, which our gain head applies to text chunks. Skim readers [23, 18, 8] skip tokens or steps within one document; we select among a row's fields, conditioned on its columns.

3 Method

Text items. Each text field is split into sentences, merged or cut into chunks of at most 48 words (chunks beyond the eighth merge into one). Items are allocated round-robin across fields up to $K = \max(8, \#\text{fields})$ per row, so every field has at least one item; the kept items contain every word of the row on 13 of the 20 datasets (Appendix B). Each item is verbalised as `passage: <column>: <chunk>` and embedded with E5-small-v2 [21] ($e_i \in \mathbb{R}^{384}$); inside the model it becomes $c_i = \text{LN}(W_{\text{item}} e_i + W_{\text{name}}\,\text{E5}(\text{field name}) + p_{\text{pos}(i)}) \in \mathbb{R}^{128}$.

Table encoder and anytime reader. Each column becomes a token $x_j w_j + m_j u_j + n_j$ (quantile-normalised numerical; m_j marks missingness) or $\text{Emb}(v_j) + n_j$ (categorical), where n_j is a projected E5 embedding of the column name. An MLP maps the tokens to $h_{\text{tab}} \in \mathbb{R}^{128}$, which initialises both layers of a GRU [3]. After reading items $a_1, \ldots, a_t$ the state is $s_t = [h_t^{(2)};\ \frac{1}{t}\sum_{\tau \le t} c_{a_\tau};\ h_{\text{tab}}]$ (the mean is zero at $t = 0$), and an MLP predicts $f(s_t)$. Training samples a fresh random reading order per batch and averages the loss over all prefixes:

$$\mathcal{L}_{\text{read}} = \tfrac{1}{2} \cdot \tfrac{1}{n+1} \textstyle\sum_{t=0}^{n} \ell\big(f(s_t), y\big) + \tfrac{1}{2}\,\ell\big(f(s_n), y\big), \tag{1}$$

with $n \le K$ the row's items and ℓ cross-entropy (classification) or MSE on the standardised target, so f is valid after *any* subset of reads.

Expected-gain policy. With the reader frozen, a head $\hat{g}(s_t, k_i) = \text{MLP}([s_t; k_i])$ regresses the observed gain of reading item i next, $\Delta_i(s_t) = \ell(f(s_t), y) - \ell(f(s_t \oplus c_i), y)$, over all unread items at states visited by an ε-greedy roll-in on held-out rows (DAgger style [17]). *Content* keys are $k_i = c_i$; *metadata* keys use only field and position, so unread content is not inspected. At inference the head needs no label: read $\arg\max_i \hat{g}$ while $\max_i \hat{g} \ge \lambda$, with λ bisected on validation rows for a target fraction of items.

TabLoRA. Joint TAR tunes E5 to the target but encodes a sentence the same way in every row. We add LoRA adapters ($r = 16$, $\alpha = 32$) to the query, key, value, attention-output and both feed-forward maps of E5's last three layers (MulTaBench's TAR placement) and gate their rank space with the row's table encoding:

$$Wx \;\mapsto\; Wx + \tfrac{\alpha}{r}\, B\,\text{diag}\big(1 + \tanh(G_W\, h_{\text{tab}})\big)\, A\, x, \tag{2}$$

with B and every G_W initialised at zero, so training starts from frozen E5 and from plain LoRA. Training has two phases: the reader is fit with frozen items (as above), then the adapters and gates are trained through $\mathcal{L}_{\text{read}}$ with the reader fixed (lr 10^{-3}, batch 64, at most 10 epochs, patience 3, fp16). Items are truncated at 128 tokens and their layer-10 inputs cached.

Table 1: Full-budget accuracy, dataset means (folds averaged per dataset). W/T/L: datasets where Ours wins, ties or loses against that row; *: two-sided Wilcoxon $p < 0.05$ (local rows only). $^{\dagger n}$: available for n datasets only; [ ]: Ours on those n.

	Classification (10, AUROC)		Regression (10, R^2)	
Method	mean	Ours W/T/L	mean	Ours W/T/L
Ours (cross-fitted readers, local heads)				
Reader, frozen E5 items	0.890	10/0/0*	0.651	10/0/0*
Reader, TabLoRA items	0.903	10/0/0*	0.672	10/0/0*
TabM Frozen + TabLoRA reader	0.910	8/2/0*	0.724	9/0/1*
TabPFN-2.5 Frozen + TabLoRA reader	0.897$^{\dagger 8}$ [0.898]	4/0/4	0.734	4/0/6
Ours: mean of the two blends	0.913		0.738	
Local baselines (identical rows)				
v1 Blend: CatBoost Frozen + reader	0.900	9/0/1*	0.702	10/0/0*
CatBoost, Joint Frozen	0.887	10/0/0*	0.684	10/0/0*
TabM, Joint Frozen	0.895	10/0/0*	0.716	10/0/0*
TabPFN-2.5, Joint Frozen	0.884$^{\dagger 8}$ [0.898]	6/0/2	0.729	5/0/5
MulTaBench reference, Joint TAR (LoRA-tuned E5)				
CatBoost, Joint TAR	0.907	9/0/1	0.716	9/0/1
LightGBM, Joint TAR	0.894	9/0/1	0.702	10/0/0
TabM, Joint TAR	0.911	5/0/5	0.728	9/0/1
TabPFN-2.5, Joint TAR	0.900$^{\dagger 8}$ [0.898]	3/0/5	0.732	4/0/6

Cross-fitted reader signals and heads. For full-budget accuracy we pair the reader with tabular learners on the Joint-Frozen frame (structured columns + PCA-30 of each field's whole-text E5 embedding). To avoid training-row leakage the reader is *cross-fitted*: three readers are trained on two thirds of the training rows each; every training row is predicted by the reader that did not see it, and test rows get the mean of the three. TabM and TabPFN-2.5 are fit on the frozen frame alone; for each, a convex weight $w \in \{0, 0.05, \ldots, 1\}$ between learner and read-all reader is tuned on a held-out 20% of the training rows, on which the reader's predictions are out-of-fold. The final prediction is the unweighted mean of the two blends; this choice and all hyperparameters were fixed before the five-split evaluation (fold-0 pilots on three datasets informed them). As in MulTaBench, TabPFN is not used beyond 10 classes (Spotify, Wine), where TabM's blend is used alone.

4 Experiments and Results

Setup. We use all 20 MulTaBench text datasets (10 classification: binary or macro one-vs-rest AUROC; 10 regression: R^2) under the paper protocol: random holdout splits (seed 42+fold, test $\leq$2,000 and train $\leq$10,000 rows) ported from the official code, five splits per dataset, on pinned dataset revisions. The reader uses one configuration everywhere ($d = 128$, dropout 0.2, AdamW lr 10^{-3}, 100 epochs, patience 15 on an inner split); TabLoRA's settings were chosen on inner-validation loss of one pilot (Jigsaw, fold 0). TabM and TabPFN-2.5 run with their default settings (TabM early-stops on the same 20% split); TabPFN-2.5's default classifier is also fine-tuned on 43 real datasets, none of them a MulTaBench dataset by name [9]. All fitting uses only the training rows of each dataset and split. TabLoRA tunes E5 on the target, so our model is target-aware and Joint TAR is its reference baseline. Local baselines are run on our exact rows; external references are the official leaderboard scores for the same split seeds (row identity unverified, so no significance test against them). Following Demšar [5], folds are averaged per dataset and two-sided Wilcoxon signed-rank tests run over datasets, separately for AUROC and R^2. Training used one T4 GPU (TabLoRA: 16 min per dataset split for all three readers).

Full-budget accuracy (Table 1). Our model improves on our previous CatBoost blend by +0.013 AUROC (9/10 datasets, $p = 0.004$) and +0.036 R^2 (10/10, $p = 0.002$), also on folds 1–4 alone, which no pilot used (+0.013, +0.039), and on Joint-Frozen TabM by +0.018 AUROC ($p = 0.002$) and +0.022 R^2 ($p = 0.002$). Over TabPFN-2.5 Frozen, the strongest local learner, the gain (+0.013, +0.009) is not significant. Against the Joint TAR references, which fine-tune E5 on the target, it is level with TabM TAR on classification (+0.002, 5/10 wins) and ahead on regression (+0.010,

Table 2: Where the gains come from: dataset-level mean difference, wins and Wilcoxon p (all five folds unless noted).

	Classification (AUROC)			Regression (R^2)		
Comparison (left − right)	Δ	wins	p	Δ	wins	p
TabLoRA vs. frozen-E5 items, reader alone	+0.013	10/10	0.002	+0.021	9/10	0.020
TabLoRA vs. frozen-E5 items, final system	+0.007	10/10	0.002	+0.001	9/10	0.049
Table gate vs. plain LoRA, reader (fold 0)	−0.001	5/10	0.625	−0.005	6/10	0.770
Adding the reader to TabM Frozen	+0.015	10/10	0.002	+0.009	9/10	0.004
Adding the reader to TabPFN-2.5 Frozen	+0.013	7/8	0.016	+0.005	8/10	0.010
Blend vs. stacking (TabM)	+0.002	6/10	0.322	+0.009	7/10	0.275
Two heads vs. TabM only	+0.003	8/10	0.008	+0.014	9/10	0.004

Table 3: Selective reading (frozen-item reader): gain policy minus a random reader with the same per-row number of reads, per nominal budget. "read": achieved test fraction; p_{Holm}: Wilcoxon p, Holm-adjusted [11] over the 8 key×budget cells per family.

		Classification (ΔAUROC)				Regression (ΔR^2)			
Keys	Budget	read	Δ	W/T/L	p_{Holm}	read	Δ	W/T/L	p_{Holm}
content	10%	10%	+0.026	9/1/0	0.031	10%	+0.051	10/0/0	0.016
content	25%	25%	+0.024	9/1/0	0.031	25%	+0.053	10/0/0	0.016
content	50%	50%	+0.014	9/1/0	0.031	50%	+0.039	10/0/0	0.016
content	75%	75%	+0.007	9/1/0	0.031	75%	+0.023	9/0/1	0.018
metadata	10%	9%	+0.010	8/1/1	0.031	10%	+0.037	10/0/0	0.016
metadata	25%	25%	+0.018	8/1/1	0.031	25%	+0.039	9/0/1	0.018
metadata	50%	54%	+0.012	8/1/1	0.031	50%	+0.034	9/0/1	0.018
metadata	75%	76%	+0.007	9/1/0	0.031	75%	+0.021	9/0/1	0.016

9/10), and −0.003 AUROC and −0.008 R^2 from the per-dataset best of the four TAR learners (Appendix A).

Ablations (Table 2). TabLoRA makes the reader itself better (+0.013 AUROC, 10/10 datasets; +0.021 R^2, 9/10), largest where the text carries the signal (Jigsaw 0.871→0.921 AUROC). After blending, half of this gain remains on classification (+0.007) and almost none on regression (+0.001), because the learners already see the frozen text embeddings. The gain comes from the adapters, not the table gate: on fold 0, gated and plain LoRA readers are level (−0.001 AUROC, −0.005 R^2, both n.s.). The reader adds to both learners, and the two blends are better than either alone: their errors differ. Stacking the reader's predictions into TabM is no better than the simpler blend.

Selective reading (Table 3). At 10% of the items the content policy beats the matched random reader by +0.026 AUROC (9/10 datasets) and +0.051 R^2 (10/10); all 16 key×budget comparisons are significant after Holm correction, and metadata keys, which never see unread content, also win at every budget. Against random items with the same *total* reads spread over rows (Appendix C), the 10% gap widens to +0.053 AUROC: choosing *which rows* to read matters as much as which items. At 10% of the items the policy recovers a median 63% of what reading everything adds over the table alone (random: 39%).

5 Conclusion

Conditioning text reading on the table helps at both levels we tried: what to read (a label-free gain policy beats budget-matched random reading at every budget) and how to read it (TabLoRA improves the reader). Cross-fitted and blended with Joint-Frozen TabM and TabPFN-2.5, the reader significantly improves on the CatBoost and TabM baselines and reaches the Joint TAR references without fine-tuning E5 on the learners' side. **Limitations:** TabLoRA's gain shrinks once strong learners are added, and its table gate added nothing over plain LoRA; the selective-reading results use the frozen-item reader; all items are embedded before selection, so the read fraction is not a compute saving; the K-item cap drops long text (Zomato). **Future work:** TAR-tuned frames for the learners, lazy item encoding, a multi-step gain policy.

References

[1] Alan Arazi, Eilam Shapira, Shoham Grunblat, Mor Ventura, Elad Hoffer, Gioia Blayer, David Holzmüller, Lennart Purucker, Gaël Varoquaux, Frank Hutter, and Roi Reichart. MulTaBench: Benchmarking multimodal tabular learning with text and image. *arXiv preprint arXiv:2605.10616*, 2026.

[2] Alan Arazi, Eilam Shapira, and Roi Reichart. TabSTAR: A tabular foundation model for tabular data with text fields. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, 2025.

[3] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014.

[4] Ian Covert, Wei Qiu, Mingyu Lu, Na Yoon Kim, Nathan White, and Su-In Lee. Learning to maximize mutual information for dynamic feature selection. In *International Conference on Machine Learning (ICML)*, 2023.

[5] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.

[6] Soham Gadgil, Ian Covert, and Su-In Lee. Estimating conditional mutual information for dynamic feature selection. In *International Conference on Learning Representations (ICLR)*, 2024.

[7] Yury Gorishniy, Akim Kotelnikov, and Artem Babenko. TabM: Advancing tabular deep learning with parameter-efficient ensembling. *arXiv preprint arXiv:2410.24210*, 2024.

[8] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

[9] Léo Grinsztajn, Klemens Flöge, Oscar Key, Felix Birkel, Philipp Jund, Brendan Roof, Benjamin Jäger, Dominik Safaric, Simone Alessi, Adrian Hayler, et al. TabPFN-2.5: Advancing the state of the art in tabular foundation models. *arXiv preprint arXiv:2511.08667*, 2025.

[10] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637:319–326, 2025.

[11] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[12] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[13] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[14] Myung Jun Kim, Léo Grinsztajn, and Gaël Varoquaux. CARTE: Pretraining and transfer for tabular learning. In *International Conference on Machine Learning (ICML)*, 2024.

[15] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

[16] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.

[17] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

[18] Minjoon Seo, Sewon Min, Ali Farhadi, and Hannaneh Hajishirzi. Neural speed reading via Skim-RNN. In *International Conference on Learning Representations (ICLR)*, 2018.

[19] Xingjian Shi, Jonas Mueller, Nick Erickson, Mu Li, and Alexander J. Smola. Benchmarking multimodal AutoML for tabular data with text fields. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.

[20] Hajin Shim, Sung Ju Hwang, and Eunho Yang. Joint active feature acquisition and classification with variable-size set encoding. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.

[21] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.

[22] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992.

[23] Adams Wei Yu, Hongrae Lee, and Quoc Le. Learning to skim text. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017.

A Per-dataset results

Table 4: Fold-mean scores per dataset. v1: CatBoost Frozen + frozen-item reader blend; TabM, PFN: Joint-Frozen learners on our rows; R / R_{LoRA}: cross-fitted reader with frozen / TabLoRA items; Ours $\pm$ fold std. LB: MulTaBench leaderboard (TabM TAR and the best of four TAR learners) for matching split seeds. Bold: best of our columns.

	Ours and local baselines (identical rows)						LB Joint TAR	
Dataset	v1	TabM	PFN	R	R_{LoRA}	Ours	TabM	best
Classification (AUROC)								
Fake Job Posting	0.911	0.920	0.935	0.892	0.924	**0.948 ± 0.010**	0.926	0.935
Jigsaw Toxicity	0.870	0.828	0.836	0.871	0.921	**0.921 ± 0.016**	0.932	0.937
Kickstarter	0.744	0.742	0.745	0.736	0.743	**0.753 ± 0.010**	0.751	0.756
DS Salary	0.831	0.833	0.837	0.825	0.830	**0.839 ± 0.004**	0.836	0.839
Michelin	0.912	0.913	0.915	0.903	0.907	**0.919 ± 0.005**	0.920	0.925
Product Sentiment	0.906	0.902	**0.917**	0.907	0.910	0.915 ± 0.005	0.919	0.927
Spotify Genres	**0.967**	0.956	–	0.953	0.958	0.966 ± 0.002	0.962	0.964
US Accidents	0.972	0.967	**0.979**	0.940	0.954	0.977 ± 0.004	0.969	0.983
Wine Review	0.979	0.977	–	0.977	0.980	**0.982 ± 0.001**	0.982	0.982
Women Clothing	0.905	0.908	0.909	0.897	0.904	**0.909 ± 0.003**	0.911	0.912
Regression (R^2)								
Babies Prices	0.875	0.896	**0.919**	0.829	0.854	0.916 ± 0.022	0.903	0.927
Book Price	0.519	0.539	**0.573**	0.376	0.463	0.571 ± 0.027	0.559	0.568
Book Readability	0.831	0.820	0.830	0.794	0.815	**0.834 ± 0.015**	0.824	0.846
Mercari	0.444	0.449	0.471	0.383	0.421	**0.479 ± 0.025**	0.471	0.472
Montgomery Sal.	0.975	0.974	**0.984**	0.962	0.969	0.982 ± 0.009	0.979	0.984
Rotten Tomatoes	0.524	0.530	0.555	0.488	0.502	**0.559 ± 0.039**	0.528	0.573
SCImago Impact	0.873	0.876	0.887	0.858	0.866	**0.891 ± 0.011**	0.880	0.892
Vancouver Sal.	0.769	0.751	**0.790**	0.749	0.753	0.787 ± 0.021	0.779	0.798
Video Games	0.381	0.488	0.411	0.447	0.431	**0.499 ± 0.112**	0.506	0.542
Zomato	0.825	0.834	**0.865**	0.621	0.649	0.861 ± 0.004	0.852	0.862

B Text coverage of the item representation

Table 5: Fold-0 test rows. Items/row: items kept after the cap K; rows over K: rows with more candidate chunks than K; words kept: share of the row's words inside kept items (an upper bound on what E5 sees).

Dataset	text fields	K	words/row	items/row	rows over K	words kept
Classification						
Fake Job Posting	3	8	176	5.4	15%	89%
Jigsaw Toxicity	1	8	52	1.8	0%	100%
Kickstarter	3	8	26	3.0	0%	100%
DS Salary	5	8	29	4.8	0%	100%
Michelin	6	8	110	7.4	17%	94%
Product Sentiment	1	8	18	1.0	0%	100%
Spotify Genres	3	8	9	3.0	0%	100%
US Accidents	6	8	19	6.0	0%	100%
Wine Review	2	8	41	2.2	0%	100%
Women Clothing	2	8	62	2.7	0%	100%
Regression						
Babies Prices	4	8	12	2.7	0%	100%
Book Price	5	8	176	7.4	43%	77%
Book Readability	6	8	185	7.8	49%	90%
Mercari	6	8	37	5.8	1%	99%
Montgomery Sal.	3	8	9	3.0	0%	100%
Rotten Tomatoes	13	13	71	12.8	1%	100%
SCImago Impact	10	10	28	10.0	0%	100%
Vancouver Sal.	2	8	5	2.0	0%	100%
Video Games	2	8	6	2.0	0%	100%
Zomato	7	8	1909	7.8	79%	6%

C Reading controls

Table 6: Frozen-item reader, dataset-level mean difference (wins/datasets) at nominal budgets 10% and 50%, all five folds. *Same total reads*: random items, with the policy's total number of reads spread over rows in proportion to their item counts. *Same per-row reads*: random items with the policy's per-row counts (as in Table 3). *Row allocation*: the second random reader minus the first. *Natural order*: fields and chunks in their stored order, same counts as the first reader. *: Holm-adjusted Wilcoxon $p < 0.05$ over the eight key×budget cells.

		Classification (ΔAUROC)		Regression (ΔR^2)	
Comparison	Keys	10%	50%	10%	50%
gain − random (same total reads)	content	+0.053 (10/10)*	+0.019 (10/10)*	+0.122 (10/10)*	+0.068 (10/10)*
	metadata	+0.020 (8/10)	+0.015 (8/10)	+0.086 (10/10)*	+0.060 (10/10)*
gain − random (same per-row reads)	content	+0.027 (9/10)*	+0.014 (9/10)*	+0.051 (10/10)*	+0.042 (10/10)*
	metadata	+0.014 (9/10)*	+0.013 (8/10)*	+0.038 (10/10)*	+0.038 (10/10)*
policy row allocation − uniform	content	+0.027 (10/10)*	+0.006 (7/10)	+0.071 (10/10)*	+0.027 (7/10)
	metadata	+0.005 (8/10)	+0.003 (4/10)	+0.048 (10/10)*	+0.022 (7/10)
gain − natural field order	content	+0.044 (10/10)*	+0.007 (7/10)	+0.123 (10/10)*	+0.086 (10/10)*
	metadata	+0.009 (6/10)	+0.003 (4/10)	+0.088 (10/10)*	+0.079 (8/10)

Table 7: Adding each text field's presence flag and log word count to the table encoder (fold 0, matched datasets). Gain − random is the advantage over the per-row matched random reader. *: Wilcoxon $p < 0.05$.

	Classification (AUROC)			Regression (R^2)		
	without	with	Δ (W/n)	without	with	Δ (W/n)
Reader, table only	0.753	0.778	+0.025 (9/10)*	0.430	0.467	+0.037 (8/10)*
Reader, read all	0.889	0.887	−0.002 (3/10)	0.671	0.645	−0.026 (2/10)
gain − random, content @10%	+0.028	+0.017	−0.011 (1/10)*	+0.051	+0.064	+0.013 (8/10)
gain − random, content @25%	+0.023	+0.017	−0.006 (1/10)*	+0.054	+0.065	+0.010 (4/10)
gain − random, metadata @10%	+0.016	+0.009	−0.007 (1/10)*	+0.034	+0.028	−0.006 (5/10)
gain − random, metadata @25%	+0.018	+0.015	−0.002 (3/10)	+0.040	+0.023	−0.017 (2/10)*